PROJECT 31A — SOFTWARE DEVELOPER ONBOARDING GUIDE (DOCUMENT 14)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

---

1. PROJECT DIRECTORY MAP

Project 31A is organized as a decoupled monorepo containing the FastAPI Python backend and the Next.js React frontend.

```
/Project 31A/
  ├── backend/                  # FastAPI Python Application
  │   ├── alembic/              # SQL schema migration revisions
  │   ├── tests/                # pytest unit and integration suites
  │   ├── pyproject.toml        # Poetry/dependency specifications
  │   └── src/
  │       └── seo_platform/
  │           ├── api/          # Route handlers, middleware, schemas
  │           ├── clients/      # External service wrapper modules
  │           ├── core/         # DB config, logs, encryption, health checks
  │           ├── llm/          # NVIDIA NIM gateway and classification logic
  │           ├── models/       # SQLAlchemy model definitions
  │           ├── services/     # Business logic modules (115 files)
  │           ├── workflows/    # Temporal Workflows and Activities
  │           ├── main.py       # API startup lifespan setup
  │           └── worker.py     # Temporal worker bootstrap
  ├── frontend/                 # Next.js React Application
  │   ├── package.json          # Node dependency mappings
  │   └── src/
  │       ├── app/              # Dashboard pages and layout routing
  │       └── lib/              # Client API client wrappers and hooks
```

---

2. CODING CONVENTIONS & DESIGN PATTERNS

Developers must adhere to these structural design patterns when extending the platform:

1. **Async-First:** All database queries, external API requests, and service interfaces must be written asynchronously using Python's `asyncio` framework.
2. **Response Envelope Enforced:** API endpoints must return data wrapped in the standard response envelope. They must never return naked database records or plain JSON dicts.
3. **No Mocks in Production:** Real API integrations must be utilized when `USE_MOCK_PROVIDERS=false`. Mock models must be isolated to `/providers/mock/` directories.
4. **Tenant Scoping Enforced:** You must never perform database queries without passing the session derived from the context manager `get_tenant_session(tenant_id)`.
5. **Durable Workflows:** Business logic that spans multiple steps or depends on external systems (e.g. outreach dispatches) must run inside Temporal Workflows.
6. **Activity Idempotency:** activities must implement Redis-backed idempotency checks to prevent duplicate dispatches or charges during retries.
7. **Strict Typings:** All python code must use static type hints (validated via `mypy`). All API request and response data must be declared as Pydantic models.

---

3. WALKTHROUGH: ADDING A NEW API ENDPOINT

Follow these steps to register a new endpoint (e.g. adding a new SEO forecasting API scope):

Step 1: Define Schemas

Create request and response models in `backend/src/seo_platform/api/schemas/forecast.py`:

```
from pydantic import BaseModel, Field


  class ForecastRequest(BaseModel):
      client_id: uuid.UUID
      target_months: int = Field(default=6, ge=1, le=24)


  class ForecastResult(BaseModel):
      month_index: int
      projected_traffic: int
      projected_value_usd: float
```

Step 2: Create the Route Handler

Create a new endpoint file `backend/src/seo_platform/api/endpoints/forecast.py`:

```
from fastapi import APIRouter, Depends
  from seo_platform.api.schemas.forecast import ForecastRequest, ForecastResult
  from seo_platform.api.middleware import get_current_user
  from seo_platform.core.database import get_tenant_session

  router = APIRouter(prefix="/forecast", tags=["Forecasting"])

  @router.post("/", response_model=APIResponseEnvelope[list[ForecastResult]])
  async def generate_seo_forecast(
      payload: ForecastRequest,
      user = Depends(get_current_user)
  ):
      async with get_tenant_session(user.tenant_id) as session:
          # Business logic goes here...
          results = await run_forecasting_logic(session, payload)
          return APIResponseEnvelope(success=True, data=results)
```

Step 3: Mount the Router

Mount the new route namespace in `backend/src/seo_platform/api/router.py`:

```
from seo_platform.api.endpoints.forecast import router as forecast_router

  # Inside register_routers()
  api_router.include_router(forecast_router)
```

---

4. WALKTHROUGH: ADDING A DATABASE MODEL

To persist new data entities (e.g. tracking Google Search Console metrics):

Step 1: Define Model

Create `backend/src/seo_platform/models/gsc_metrics.py`:

```
from sqlalchemy import Column, String, Integer, ForeignKey
  from seo_platform.models.base import Base, UUIDPrimaryKeyMixin, TenantMixin, Timestam
  pMixin

  class GscMetric(Base, UUIDPrimaryKeyMixin, TenantMixin, TimestampMixin):
      __tablename__ = "gsc_metrics"

      client_id = Column(UUID(as_uuid=True), ForeignKey("clients.id", ondelete="CASCAD
  E"), nullable=False)
      keyword = Column(String(500), nullable=False)
      impressions = Column(Integer, default=0, nullable=False)
      clicks = Column(Integer, default=0, nullable=False)
```

Step 2: Register in Model Package

Import the new model class inside `backend/src/seo_platform/models/__init__.py` to expose it to Alembic auto-discovery:

```
from seo_platform.models.gsc_metrics import GscMetric
```

Step 3: Run alembic Migrations

Run the generation script inside the terminal:

```
alembic revision --autogenerate -m "add_gsc_metrics_table"
  alembic upgrade head
```

---

5. WALKTHROUGH: EXTENDING TEMPORAL ACTIONS

To register a new task step in a campaign workflow:

Step 1: Define the Activity

Activities contain the actual side-effects (e.g. sending a Slack notification). Create the definition:

```
from temporalio import activity

  @activity.defn(name="send_slack_alert_activity")
  async def send_slack_alert_activity(tenant_id: str, channel: str, message: str) -> bo
  ol:
      idempotency_key = f"slack:{tenant_id}:{hash(message)}"
      if await idempotency_store.exists(idempotency_key):
          return True

      result = await slack_client.send_message(channel, message)
      await idempotency_store.set(idempotency_key, "sent", ttl=86400)
      return result
```

Step 2: Register in Worker Bootstrap

Register the new activity in `backend/src/seo_platform/worker.py`:

```
# Import the activity definition
  from seo_platform.workflows.slack import send_slack_alert_activity

  # Add to the activities list passed to the Temporal Worker initialization
  worker = Worker(
      client,
      task_queue="BACKLINK_ENGINE",
      workflows=[BacklinkCampaignWorkflow],
      activities=[discover_prospects_activity, ..., send_slack_alert_activity]
  )
```

Step 3: Call inside the Workflow Definition

Workflows invoke activities using proxy references:

```
# Inside BacklinkCampaignWorkflow.run()
  await workflow.execute_activity(
      send_slack_alert_activity,
      args=[str(self.tenant_id), "#alerts", f"Campaign {self.name} launched!"],
      task_queue="BACKLINK_ENGINE",
      start_to_close_timeout=timedelta(seconds=60),
      retry_policy=RetryPreset.EXTERNAL_API
  )
```

---

6. ERROR HANDLING & LOGGING CONVENTIONS

6.1 Structured Error Exceptions

All internal service errors must inherit from `PlatformError` defined in `backend/src/seo_platform/core/errors.py`.

```
class PlatformError(Exception):
      def __init__(self, error_code: str, message: str, details: dict = None, retryabl
  e: bool = False):
          self.error_code = error_code
          self.message = message
          self.details = details or {}
          self.retryable = retryable
```

6.2 Structured Logging with structlog

Use structured logging to ensure searchable JSON outputs in production logs:

```
import structlog
  logger = structlog.get_logger(__name__)

  # Correct Pattern: Pass variables as key-value keyword arguments
  logger.info(
      "prospecting_started",
      tenant_id=str(tenant_id),
      campaign_id=str(campaign_id),
      prospects_count=len(prospects)
  )
```

Avoid string interpolation (f-strings) inside log messages, as this prevents index parsers (Elasticsearch, Loki) from indexing variable keys.